

UD 6. Webs híbridas. Servicios Web

Índice de contenidos

1. Servicios web.....	2
1.1. ¿Qué es?.....	2
1.2. Características.....	3
2. SOAP.....	5
2.1. ¿Cómo funciona?.....	5
2.2. Descripción del servicio: WSDL.....	6
3. API REST.....	8
3.1. ¿Cómo funciona?.....	8
3.2. Consumir API REST.....	9
3.2.1 Consumir GET.....	10
3.2.2 Consumir POST.....	12
3.2.3 Consumir PUT.....	14
3.2.4 Consumir DELETE.....	16
3.3. Implementar API REST.....	17
3.3.1 Implementar GET.....	17
3.3.2 Implementar POST.....	20
3.3.3 Implementar PUT.....	22
3.3.4 Implementar DELETE.....	23

1. Servicios web

1.1. ¿Qué es?

En ocasiones, las aplicaciones que desarrolles necesitarán compartir información con otras aplicaciones.

Sin ir más lejos, cojamos de ejemplo una aplicación de tienda web. La información que se almacena sobre los productos incluye su código, nombre, descripción, PVP, etc. Seguramente los proveedores a los que se compran los artículos, manejen la misma o parecida información. Y quizás puedas aprovechar esa información para tu propia aplicación.

O puede ser que, una vez que esté finalizada y funcionando, quieras programar una nueva aplicación (y no necesariamente una aplicación web) que la complemente para, por ejemplo, procesar la información sobre los pedidos realizados.

Para compartir la información que gestiona tu aplicación, normalmente es suficiente con dar acceso a la base de datos en que se almacena. Pero ésta generalmente no es una buena idea. Cuantas más aplicaciones utilicen los mismos datos, más posibilidades hay de que se generen errores en los mismos.

Por otro lado, gran parte de la información que gestionan las aplicaciones web ya está disponible para que otros la utilicen (dejando a un lado las consideraciones relacionadas con el control de acceso). Por ejemplo, si alguien quiere conocer el precio de un producto en la tienda web, basta con buscar ese producto en la página en que se listan todos los productos. Pero, para que esa misma información (el precio de un producto) la pueda obtener un programa, éste tendría que contemplar un procedimiento para buscar el producto concreto dentro de las etiquetas HTML de la página y extraer su precio (proceso conocido como **Web Scrapping**).

Para facilitar esta tarea existen los servicios web. Un **servicio web** es un método que permite que dos equipos intercambien información a través de una red informática. Al utilizar servicios web, el servidor puede ofrecer un punto de acceso a la información que quiere compartir. De esta forma controla y facilita el acceso a la misma por parte de otras aplicaciones.

Volviendo al ejemplo de nuestra tienda, si quisiéramos aprovechar la información de que disponen nuestros proveedores, éstos tendrían que ofrecer un servicio web que nos permitiese recuperarla. Por ejemplo, enviándonos el código de un producto, podríamos obtener su nombre, descripción, precio, etc. Inversamente, si quisiéramos facilitar la obtención de datos de nuestra tienda por parte de otras aplicaciones, podríamos programar y ofrecer un servicio web de forma que, por ejemplo, devolviese el listado de pedidos del cliente que se requiera.

1.2. Características

Existen numerosos protocolos que permiten la comunicación entre ordenadores a través de una red: FTP, HTTP, SMTP, POP3, TELNET, etc. En todos estos protocolos se definen un servidor y un cliente. El **servidor es la máquina que está esperando conexiones (escuchando)** por parte de un cliente. El **cliente es la máquina que inicia la comunicación**. Cada uno de estos protocolos tiene asignado además un puerto (TCP o UDP) concreto, que será el que utilicen normalmente los equipos servidores.

Cada uno de los protocolos que hemos nombrado ha sido creado para un fin específico: FTP para transferencia de archivos, HTTP para páginas web, SMTP y POP3 para correo electrónico, y TELNET para acceso remoto. No han sido diseñados para transportar peticiones de información genéricas entre aplicaciones, como solicitar el PVP de un producto. Sin embargo, ya desde hace tiempo existen otras soluciones para este tipo de problemas. Una de las más populares es **RPC**.

El **protocolo RPC** se creó para permitir a un sistema acceder de forma remota a funciones o procedimientos que se encuentren en otro sistema. El cliente se conecta con el servidor, y le indica qué función debe ejecutar. El servidor la ejecuta y le devuelve el resultado obtenido. Así, por ejemplo, podemos crear en el servidor RPC una función que reciba un código de producto y devuelva su precio.

RPC usa su propio puerto, pero normalmente solo a modo de directorio. Los clientes se conectan a él para obtener el puerto real del servicio que les interesa. Este puerto no es fijo; se asigna de forma dinámica.

Los **servicios web se crearon para permitir el intercambio de información al igual que RPC**, pero sobre la base del protocolo HTTP (de ahí el término web). En lugar de definir su propio protocolo para transportar las peticiones de información, utilizan HTTP para este fin. La respuesta obtenida no será una página web, sino la información que se solicitó. De esta forma pueden funcionar sobre cualquier servidor web; y, lo que es aún más importante, utilizando el puerto 80 reservado para este protocolo. Por tanto, cualquier ordenador que pueda consultar una página web, podrá también solicitar información de un servicio web. Si existe algún cortafuegos en la red, tratará la petición de información igual que lo haría con la solicitud de una página web.

Las opciones más utilizadas actualmente para ofrecer servicios web son:

- SOAP**: Protocolo de mensajes que utiliza XML para intercambiar información. Se utiliza junto a WSDL (un formato que permite describir servicios web).
- API REST**: Arquitectura de comunicación basada en el protocolo HTTP. Cada elemento tiene su propia URI a través de la cual podemos obtener sus datos. Se suele utilizar XML o JSON en sus comunicaciones. Es la más usada en la actualidad.
- GraphQL**: protocolo de API en el que el usuario proporciona una estructura vacía y el servidor se la devuelve rellena con los datos solicitados.

En esta unidad nos centraremos en SOAP y API REST.

Para saber más ...

En esta [comparación de Google Trends](#) puedes comprobar cómo ha evolucionado la búsqueda en Google de los términos `xml api`, `json api` y `GraphQL`

2. SOAP

2.1. ¿Cómo funciona?

SOAP es un protocolo que indica cómo deben ser los mensajes que se intercambian entre el servidor y el cliente, cómo deben procesarse éstos, y cómo se relacionan con el protocolo que se utiliza para transportarlos de un extremo a otro de la comunicación (en el caso de los servicios web, este protocolo será HTTP).

Aunque nosotros vamos a utilizar HTTP para transmitir la información, SOAP no requiere el uso de un protocolo concreto para transmitir la información. SOAP se limita a definir las reglas que rigen los mensajes que se deben intercambiar el cliente y el servidor. Cómo se envíen esos mensajes no es relevante desde el punto de vista de SOAP. En lugar de utilizar HTTP para transmitirlos, se podrían utilizar, por ejemplo, correos electrónicos (claro que en este caso ya no sería un servicio web).



Veamos un ejemplo. Si implementamos un servicio web para informar sobre el precio de los artículos que se venden en la tienda web, una petición de información para el artículo con código 'KSTMSDHC8GB' podría ser de la siguiente forma:

```
<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope
  xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:ns1=""
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:soap-enc="http://schemas.xmlsoap.org/soap/encoding/"
  soap:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/"
>
  <soap:Body>
    <ns1:getPVP>
      <param0 xsi:type="xsd:string">KSTMSDHC8GB</param0>
    </ns1:getPVP>
  </soap:Body>
</soap:Envelope>
```

Y su respectiva respuesta:

```
<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope
  xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:ns1=""
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:soap-enc="http://schemas.xmlsoap.org/soap/encoding/"
  soap:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/"
>
  <soap:Body>
    <ns1:getPVPResponse>
      <return xsi:type="xsd:string">10.20</return>
    </ns1:getPVPResponse>
  </soap:Body>
</soap:Envelope>
```

2.2. Descripción del servicio: WSDL

Una vez que hayas creado un servicio web, puedes programar el correspondiente cliente y comenzar a utilizarlo. Como el servicio lo has creado tú, sabrás cómo acceder a él: en qué URL está accesible, qué parámetros recibe, cuál es la funcionalidad que aporta, y qué valores devuelve. Sin embargo, si lo que quieres es que el servicio web sea accesible a aplicaciones desarrolladas por otros programadores, deberás indicarles cómo usarlo, es decir, crear un documento WSDL que describa el servicio.

WSDL es un lenguaje basado en XML que utiliza unas reglas determinadas para generar el documento de descripción de un servicio web. Una vez generado, ese documento se suele poner a disposición de los posibles usuarios del servicio (normalmente se accede al documento WSDL añadiendo **?wsdl** a la URL del servicio).

La estructura de un documento WSDL es la siguiente:

```
<definitions
  name="..."
  targetNamespace="http://..."
  xmlns:tns="http://..."
  xmlns="http://schemas.xmlsoap.org/wsdl/"
  ...
>
  <types>
    ...
  </types>
  <message>
    ...
  </message>
  <portType>
    ...
  </portType>
  <binding>
    ...
  </binding>
  <service>
    ...
  </service>
</definitions>
```

El objetivo de cada una de las secciones del documento es el siguiente:

- **types.** Incluye las definiciones de los tipos de datos que se usan en el servicio.
- **message.** Define conjuntos de datos, como la lista de parámetros que recibe una función o los valores que devuelve.
- **portType.** Cada portType es un grupo de funciones que implementa el servicio web. Cada función se define dentro de su portType como una operación (operation).
- **binding.** Define cómo va a transmitirse la información de cada portType.
- **service.** Contiene una lista de elementos de tipo port. Cada port indica dónde (en qué URL) se puede acceder al servicio web.

Veamos de ejemplo el siguiente WSDL correspondiente a un servicio web de una calculadora:

<http://www.dneonline.com/calculator.asmx?WSDL>

3. API REST

3.1. ¿Cómo funciona?

Una **API (Application Programming Interface)** es una interfaz que los desarrolladores de una aplicación proporcionan para que otros desarrolladores puedan interactuar con esta, ya sea obteniendo información (obtener datos de un elemento, un listado de ellos) o enviando órdenes (añadir, modificar, borrar...).

Las **API REST (Representational State Transfer)** son un tipo de APIs que cumplen una serie de requisitos:

- **Ofrecen un protocolo cliente/servidor sin estado:** esto quiere decir que todas las peticiones las realiza un cliente (el usuario de la API) a un servidor y cada una es independiente de las demás (no se recuerda nada de peticiones anteriores, no hay estados).
- **Todos los recursos se manipulan a partir de una única URI o endpoint:** todos los objetos tienen una URI que los identifica y esta siempre es la misma independientemente de la acción que queramos realizar.

Ejemplos:

```
/noticias
/noticias/23
/clientes
/clientes/42
/clientes/42/facturas/6
```

- **Las operaciones se indican según el método HTTP utilizado en la petición** y se hace uso de los códigos de respuesta para indicar el resultado:

- **POST:** se utiliza para enviar datos al servidor.

POST /clientes creará un cliente con los datos suministrados por parámetros. Devolverá el cliente creado o un enlace a este y el código 201 (CREATED) si todo ha ido bien.

- **GET:** se utiliza para leer y consultar datos del servidor.

GET /clientes devolverá un listado de URIs de clientes.

GET /clientes/25 devolverá los datos del cliente 25.

- **PUT:** se utiliza para actualizar los datos en el servidor.

PUT /clientes/25 modificará los datos del cliente 25 con los datos aportados por parámetros. Devolverá los datos actualizados del cliente 25 o un enlace a estos.

- **DELETE:** se utiliza para eliminar datos en el servidor.

DELETE /clientes/25 borrará el cliente 25. Si todo va bien la respuesta estará vacía y devolverá el código 204 (NO CONTENT).

- **PATCH:** se utiliza para cambiar algún dato. Similar a PUT, solo que no se envía toda la información del objeto, sino únicamente la que se quiere modificar. Esta operación se utiliza muy poco.

Para saber más ...

En [esta página](#) puedes encontrar los códigos de respuesta HTTP más utilizados en APIs REST.

No hay un estándar que defina claramente cómo debe comportarse una **API REST**, de modo que los códigos de respuesta, los parámetros, la estructura de la información que devuelve... suele cambiar de una API a otra. Debido a esto, las APIs **suelen contar con documentación** en la que explican su uso.

El uso de una **API REST** ofrece las siguientes ventajas:

- **Separación entre datos e interfaz:** el servidor se dedica únicamente a gestionar los datos, mientras que la interacción con el usuario corre a cargo del cliente.
- **Visibilidad, fiabilidad y escalabilidad:** estos sistemas son fáciles de usar, integrar, probar y escalar.
- **Independencia del tipo de plataformas o lenguajes:** por ejemplo, una aplicación móvil y una web pueden trabajar con el mismo servidor y estar escritos en cualquier lenguaje.

3.2. Consumir API REST

Para realizar peticiones (ya sean GET, POST, PUT o DELETE) a un servicio web ya existente, necesitamos un cliente HTTP. Los navegadores web son clientes HTTP, pero nosotros necesitamos un cliente HTTP para usarlo en el código PHP. Existen muchas librerías de clientes HTTP como Guzzle, Buzz, HTTPPlug... Nosotros usaremos el cliente HTTP propio que viene con Laravel y que está basado en el popular Guzzle.

3.2.1 Consumir GET

Como ejemplo, vamos a utilizar una API pública que muestra información sobre películas. Es <https://www.omdbapi.com/>.

Esta API requiere registro, para limitar cuántas peticiones vamos a poder realizar al día y no saturar la página. Una vez registrado, recibiremos una **api-key** en nuestro email.

Nos indica que para realizar una petición tendremos que usar [http://www.omdbapi.com/?apikey=\[yourkey\]&](http://www.omdbapi.com/?apikey=[yourkey]&) donde [yourkey] lo sustituiremos por nuestra api-key.

Si leemos la web, podemos ver todas las opciones que existen para consultar las películas. Por ejemplo el parámetro “s” sirve para indicarle el nombre de una película.

Un ejemplo de petición GET para obtener todas las películas cuyo nombre contenga “Titanic” es:

[http://www.omdbapi.com/?apikey=\[api-key\]&s=Titanic](http://www.omdbapi.com/?apikey=[api-key]&s=Titanic)

(Importante: debes sustituir [api-key] por tu api-key recibida al registrarte).

En el siguiente código se lanza una petición GET utilizando **Http** de Laravel:

```
<?php
namespace App\Http\Controllers;

use Illuminate\Support\Facades\Http;

class ConsumirRestApiController extends Controller
{
    public function listarPelículas()
    {
        $respuesta = Http::get('http://www.omdbapi.com/?apikey=TUAPIKEY&s=Titanic');

        //Comprobamos si la respuesta ha sido exitosa (si ha devuelto un código entre 200 y 299)
        if ($respuesta->successful()) {
            // Convertir la respuesta en JSON a un array asociativo
            $películas = $respuesta->json();

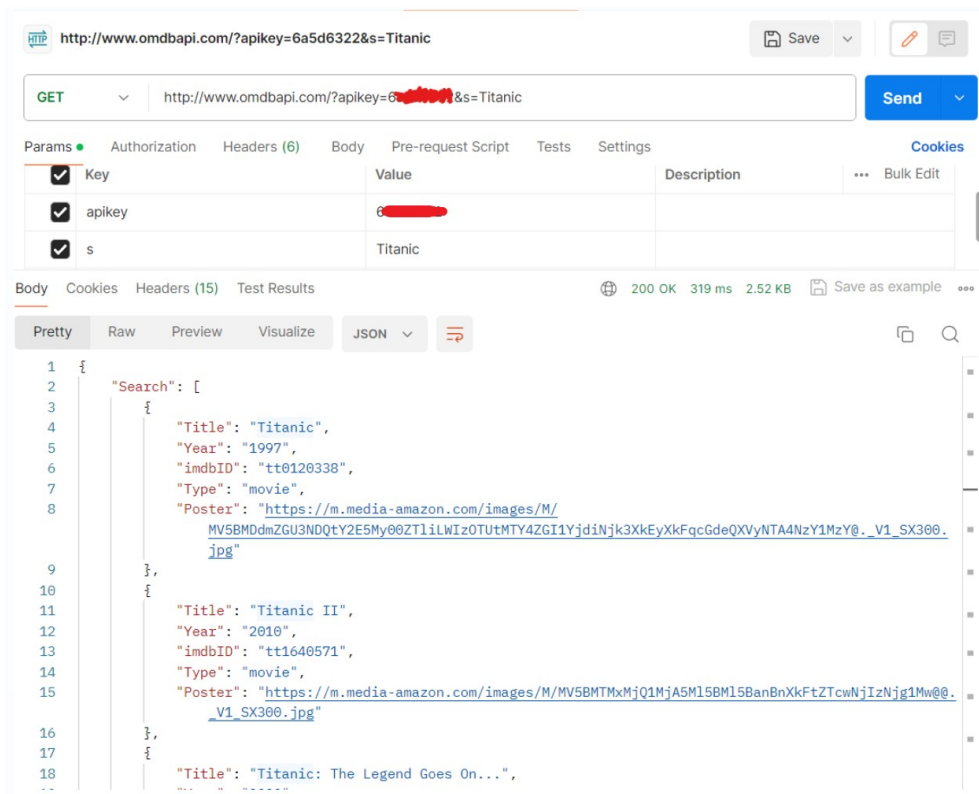
            //El formato de películas es así:
            //[
            //    ["Title" => "Titanic", "Year" => "1997", "imdbID" => "tt0120338"],
            //    ["Title" => "Titanic II", "Year" => "2010", "imdbID" => "tt1640571"],
            //    ["Title" => "Titanic", "Year" => "1953", "imdbID" => "tt0330994"]
            //]

            //Haciendo esto ya me quedaría un array con arrays asociativos dentro.
            $películas = $películas["Search"];

            return view("listaPelículas", ["películas" => $películas]);
        } else {
            //Mostramos una plantilla indicando de que ha ocurrido un error
            return view("error");
        }
    }
}
```

Como se puede ver, la parte más difícil es descifrar el formato de la respuesta para poder saber cómo presentarla. La respuesta en crudo de las películas era un array asociativo que contenía arrays numéricos y estos a su vez, un array asociativo.

Es por ello por lo que es importante **comprobar previamente** el formato utilizado en la respuesta mediante el uso de un cliente REST como POSTMAN:



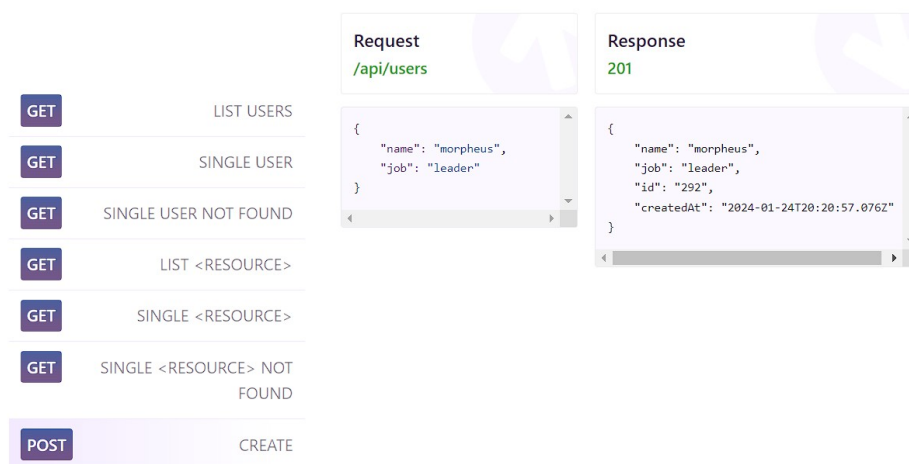
Actividad 6.0

Crea un nuevo método en el controlador. Este método deberá realizar una petición GET para obtener únicamente la película de Titanic lanzada en 2022. Mostrar sus datos (incluyendo su imagen) en una plantilla.

3.2.2 Consumir POST

Para consumir POST utilizaremos de ejemplo otra API pública que permite realizar operaciones de modificación (POST, PUT, DELETE). La anterior API (la de las películas) solo permitía la operación GET.

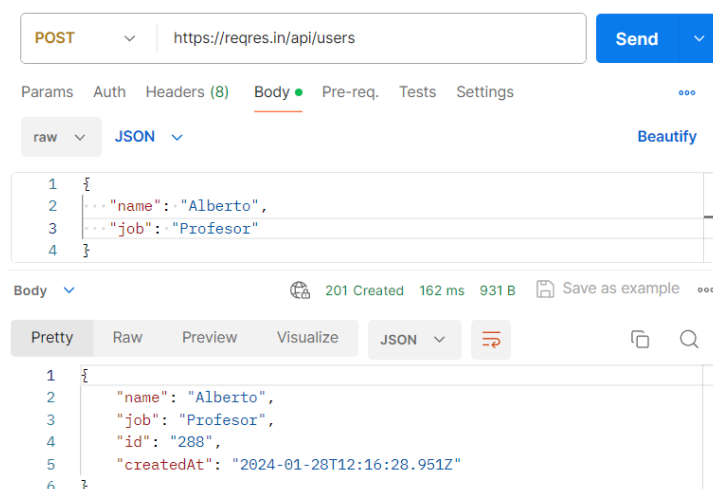
Se utilizará <https://reqres.in/>. Esta API gestiona datos de personas y no requiere registro. Si accedemos a su web, podemos ver los ejemplos de uso para POST:



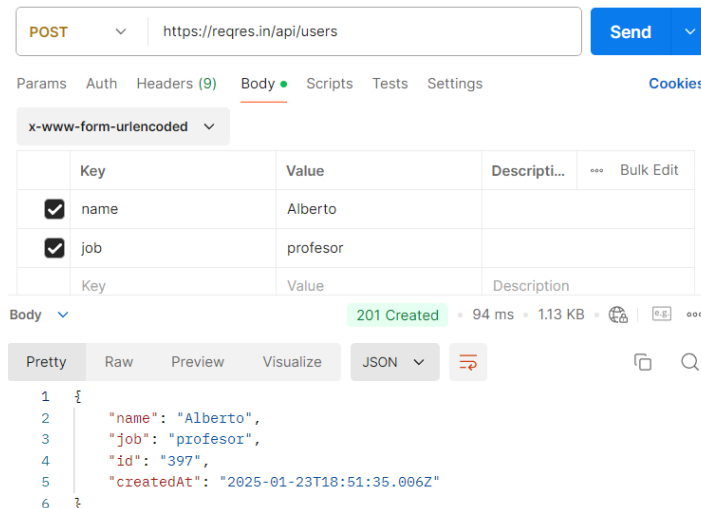
Donde pone Request, nos indica cuál es la URI para realizar la petición de crear una nueva persona (<https://reqres.in/api/users>).

También nos indica el formato del **cuerpo**. El cuerpo son los datos que le subiremos a la API. En este caso, serán los datos de la persona que vamos a crear. Podemos ver como usa el formato **JSON**. Para crear una nueva persona, necesitaremos mandarle en el cuerpo dos datos: el nombre de la persona (**name**) y su trabajo (**job**).

Podemos probar a enviar una solicitud POST utilizando Postman. Tenemos que seleccionar la operación (**POST**), el tipo de dato que se enviará (**raw**), concretamente siendo **JSON**.



Hay muchas APIs donde el cuerpo de la petición no lo podemos enviar en formato JSON, sino que tenemos que usar el formato **x-www-form-urlencoded**. Esta API que estamos usando también permite este formato tal y como se puede observar aquí:



Tras enviar la solicitud, nos devuelve un código **201** que representa que la petición POST se ha realizado con éxito. También nos devuelve otros datos como la **id** y la fecha de creación (**createdAt**).

Ahora se mostrará el código utilizando el cliente en PHP:

```
public function crearPersona()
{
    $datosCuerpo = [
        "name" => "Alberto",
        "job" => "Profesor"
    ];

    //Realizamos la petición y obtenemos la respuesta en crudo (formato JSON)
    $respuesta = Http::post('https://reqres.in/api/users', $datosCuerpo);

    //Si todo ha ido bien (código 201 OK)
    if ($respuesta->successful()) {

        //Convertimos el JSON en crudo de la respuesta en un array asociativo
        $respuesta = $respuesta->json();

        //El formato de la respuesta es así:
        //[
        //  ["name" => "Alberto", "job" => "Profesor", "id" => "816", "createdAt" => "2024-01-24T20:23:11.734Z"],
        //]

        //Nos quedamos únicamente con la fecha de creación
        $fechaCreacion = $respuesta["createdAt"];

        return view("personaCreada", ["fechaCreacion" => $fechaCreacion]);
    } else {
        return view("error");
    }
}
```

Observa las diferencias respecto a la petición GET que hicimos en el apartado anterior:

- Ahora en la clase Http usamos “**post**” en lugar de “get”.
- El cuerpo contiene los datos a enviar en la petición. Lo hemos codificado en un array asociativo (**\$datosCuerpo**).
- El método Http::post() tiene un tercer parámetro donde le pasamos el cuerpo.

Actividad 6.1

Crea un nuevo método en el controlador. Este método deberá realizar una petición POST para almacenar un producto en la API <https://restful-api.dev/>. El producto tendrá como nombre (name) “Tostadora”. Tendrá además dos atributos extra (data): “marca” cuyo valor será “Smeg” y “color” cuyo valor será “rojo”. Muestra en una plantilla el id que se genere para el producto guardado. Prueba a hacer una petición GET utilizando Postman para comprobar que efectivamente se ha guardado.

3.2.3 Consumir PUT

Para consumir la operación PUT seguiremos utilizando de ejemplo <https://reqres.in/>.

Esta operación es una combinación de GET y POST. Por un lado indicaremos en la **URI** el recurso que queremos actualizar (en nuestro caso, el id de la persona) y por otro lado enviaremos el **cuerpo** que contiene los nuevos datos de la persona.

Importante: siempre que enviemos una petición PUT, debemos indicar en el cuerpo todos los datos. Por ejemplo, si queremos modificar solamente el trabajo de persona, tenemos que enviar también su nombre.

Fíjate como la página web de la API nos indica cómo escribir la URI. Será https://reqres.in/api/users/{id_de_la_persona}

The screenshot displays a REST client interface with a list of HTTP methods on the left and a detailed view of a PUT request and response on the right.

Method	Action
GET	LIST USERS
GET	SINGLE USER
GET	SINGLE USER NOT FOUND
GET	LIST <RESOURCE>
GET	SINGLE <RESOURCE>
GET	SINGLE <RESOURCE> NOT FOUND
POST	CREATE
PUT	UPDATE

Request	Response
<code>/api/users/2</code>	<code>200</code>
<pre>{ "name": "morpheus", "job": "zion resident" }</pre>	<pre>{ "name": "morpheus", "job": "zion resident", "updatedAt": "2024-01-28T12:36:53.020Z" }</pre>

Se va a modificar la persona creada en el apartado anterior con otro trabajo:

```
public function actualizarPersona()
{
    $datosCuerpo = [
        "name" => "Alberto",
        "job" => "Director"
    ];

    //Realizamos la petición y obtenemos la respuesta en crudo (formato JSON)
    $respuesta = Http::put('https://reqres.in/api/users/288', $datosCuerpo);

    //Si todo ha ido bien (código 201 OK)
    if ($respuesta->successful()) {

        //Convertimos el JSON en crudo de la respuesta en un array asociativo
        $respuesta = $respuesta->json();

        //Nos quedamos únicamente con la fecha de actualización
        $fechaCreacion = $respuesta["updatedAt"];

        return view("personaCreada", ["fechaCreacion" => $fechaCreacion]);
    } else {
        return view("error");
    }
}
```

Observa las diferencias respecto a la petición POST que hicimos en el apartado anterior:

- Se ha modificado el array \$datosCuerpo con el nuevo trabajo de Alberto.
- Ahora en la clase Http usamos “**put**”.
- En la URI de la petición se ha indicado el id de la persona que se quiere modificar. Se utiliza el id 288 porque es la persona que creamos en el apartado anterior.

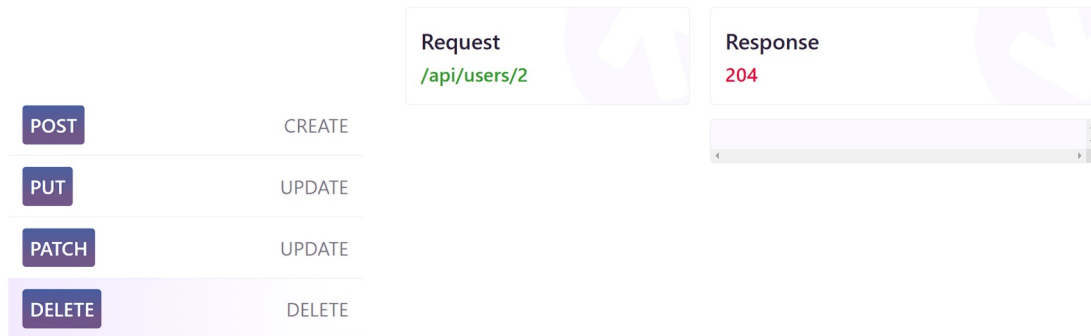
Actividad 6.2

Crea un nuevo método en el controlador. Este método deberá modificar el producto creado en la actividad anterior, cambiándole el color de la Tostadora a “negra”. Muestra en una plantilla la fecha y hora de la actualización. Prueba a hacer una petición GET utilizando Postman para comprobar que efectivamente se ha modificado.

3.2.4 Consumir DELETE

Para consumir la operación DELETE seguiremos utilizando de ejemplo <https://reqres.in/>.

Tal y como podemos ver en la web de la API, lo único que hay que tener en cuenta a la hora de realizar la solicitud DELETE es indicar el id del recurso a borrar en la URI. Además, podemos observar como a diferencia de las otras operaciones, esta no devolverá nada especial en la respuesta (simplemente el código 204).



Implementémoslo en el código:

```
public function borrarPersona()
{
    //Realizamos la petición y obtenemos la respuesta en crudo (formato JSON)
    $respuesta = Http::delete('https://reqres.in/api/users/288');

    //Si todo ha ido bien (código 204 OK)
    if ($respuesta->successful()) {
        return view("personaBorrada");
    } else {
        return view("error");
    }
}
```

Actividad 6.3

Crea un nuevo método en el controlador en la ruta /borrarproducto/{id}. Este método deberá realizar una petición DELETE para borrar un producto en la API <https://restful-api.dev/>. Indica en la plantilla el mensaje obtenido al borrarlo. Prueba a hacer una petición GET utilizando Postman para comprobar que efectivamente se ha borrado.

3.3. Implementar API REST

Aunque existen formas de automatizar esta tarea, vamos a implementar una API REST de forma manual.

Utilizaremos de ejemplo el proyecto de Laravel “**tweetravel**”, el cuál intenta simular la plataforma de Twitter. Solo tiene dos entidades: Tweet y User.

Puedes descargar el proyecto desde Moodle. Una vez descargado, sitúate con la terminal en el proyecto y realiza lo siguiente:

1. Ejecutar **composer install** para instalar las dependencias software.
2. Personalizar la configuración de la base de datos en el archivo **.env**.
3. Ejecutar **php artisan migrate** para crear las tablas en la base de datos.
4. Ejecutar **php artisan db:seed** para cargar los datos de ejemplo en la BD.

Ya con el proyecto configurado y en funcionamiento podemos seguir con la implementación de las operaciones API REST. Gestionaremos los tweets y usuarios: crear tweets, modificar tweets, obtener información de usuarios, borrar usuarios...

Todos los recursos estarán dentro de la ruta **api/** del siguiente modo:

- Tweets: **/api/tweets**
- Usuarios: **/api/users**

Antes de comenzar, vamos a crear un controlador que se encargue de todo lo relacionado con la API. Le llamaremos ApiController:

```
php artisan make:controller ApiController
```

3.3.1 Implementar GET

Vamos a empezar implementando la petición GET para obtener la información de un tweet y un usuario. Para ello, primero crearemos las URI a las que accederán los usuarios que quieran obtener información de nuestra aplicación a través de la API. Es decir, crearemos las rutas en el archivo *web.php*:

```
Route::get('/api/users/{id}', [ApiController::class, 'getUser'])->name('getUser');  
Route::get('/api/tweets/{id}', [ApiController::class, 'getTweet'])->name('getTweet');
```

Vamos a implementar `getUser($id)` en el controlador `ApiController`. Para ello tenemos que:

1. Obtener el User con dicha `$id` desde la base de datos. Si el usuario no existe, deberemos devolver un error 404 (not found).
2. En caso de que exista, crearemos un array con los datos que queramos devolver. En lugar de devolver la información de los tweets del usuario, devolveremos los enlaces de los tweets.
3. Devolvemos el objeto respuesta en formato JSON.

El código quedaría de la siguiente forma:

```
public function getUser($id)
{
    $user = User::find($id);

    if ($user == null) {
        //Creo array asociativo con el mensaje que quiero devolver.
        $data = ['error' => 'User not found'];

        //Devuelvo la respuesta con el mensaje y el código 404 (NOT FOUND).
        return response()->json($data, 404);
    }
    else {
        //Creo array asociativo vacío y lo relleno con los datos del usuario
        $data = [];
        $data["id"] = $user->id;
        $data["name"] = $user->name;
        $data["userName"] = $user->userName;

        //Creo un array numérico donde almacenaré los enlaces API de
        //los tweets que pertenecen al usuario.
        $tweets = [];

        foreach ($user->tweets as $tweet) {
            //route() está generando "http://localhost:8000/api/tweets/id"
            $tweets[] = route("getTweet", ["id" => $tweet->id]);
        }

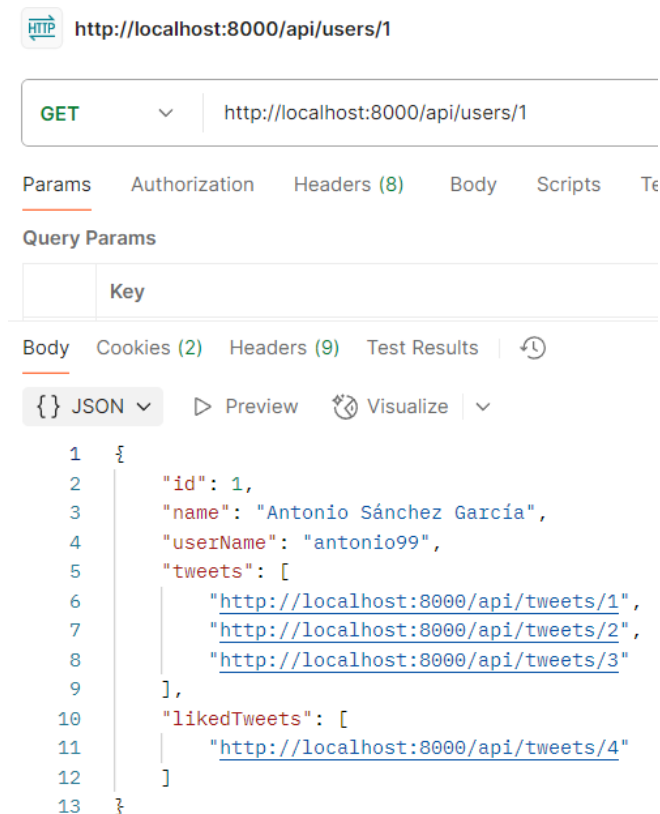
        //Añadimos los tweets con la clave "tweets" a los datos
        //finales que se van a devolver.
        $data["tweets"] = $tweets;

        //Creo un array numérico donde almacenaré los enlaces API de los
        //tweets a los que le ha dado like el usuario.
        $likedTweets = [];
        foreach ($user->likedTweets as $tweet) {
            $likedTweets[] = route("getTweet", ["id" => $tweet->id]);
        }

        //Añadimos los tweets con la clave "tweets" a los datos finales que se van a devolver.
        $data["likedTweets"] = $likedTweets;

        //Devuelvo la respuesta con el mensaje y el código 200 (OK).
        return response()->json($data, 200);
    }
}
```

Podemos probar que la API funciona lanzando una petición GET desde Postman a `http://localhost:8000/api/users/1` por ejemplo:



Actividad 6.4

Implementa el método `getTweet($id)` que obtenga información del tweet.

Implementa el método `getTweets()` que devolverá un listado de todos los tweets. Este método debe implementar la petición GET en `/api/tweets`.

Implementa el método `getUsers()` que devolverá un listado de todos los usuarios. Este método debe implementar la petición GET en `/api/users`.

3.3.2 Implementar POST

Vamos a añadir la funcionalidad POST a la ruta `/api/users/`.

```
Route::post('/api/users', [ApiController::class, 'createUser'])->name('createUser');
```

Para añadir un usuario vamos a necesitar que nos envíen a través de la petición POST los siguientes parámetros:

- `name`: Nombre del usuario a añadir
- `userName`: nombre de usuario del usuario a añadir. Debe ser único.

Vamos a implementar `createUser()`. Para ello tenemos que:

1. Comprobar que el `userName` no exista ya en la base de datos. Si existe, deberemos devolver un error 409 (Conflicto).
2. Crear un objeto de la clase `User`, rellenarlo con la información recibida en la petición y guardarlo en la base de datos.
3. Devolver un mensaje con código 201 y una respuesta. Tenemos dos opciones para enviar la respuesta:
 - Devolver toda la información del usuario creado (`id`, `name`, `userName`...).
 - Un enlace al usuario (`localhost:8000/api/users/{id}`).

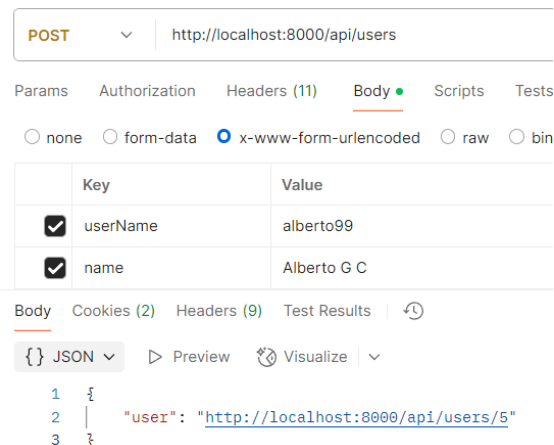
El código, devolviendo un enlace, quedaría de la siguiente forma:

```
public function createUser(Request $r) {
    //Comprobar que no exista en la BD un usuario con el mismo userName
    $user = User::where('userName', '=', $r->get("userName"))->first();

    if ($user != null) {
        $data = ['error' => 'UserName already exists'];
        return response()->json($data, 409);
    }
    else {
        //Crear objeto User y rellenarlo con los datos de la petición
        $user = new User();
        $user->name = $r->get("name");
        $user->userName = $r->get("userName");
        //Guardar objeto en la BD
        $user->save();

        //Enviar respuesta
        $data = ["user" => route("getUser", ["id" => $user->id])];
        return response()->json($data, 201);
    }
}
```

Podemos probarlo con Postman:



Es posible que al intentar probarlo con Postman te devuelva **error 419**. Eso ocurre cuando la verificación CSRF falla. Para solucionarlo, vamos a decirle a Laravel que no intente verificar las rutas que comiencen por `/api`. Para ello, editamos el archivo ***bootstrap/app.php*** y añadimos la línea que aparece en negrita:

```
->withMiddleware(function (Middleware $middleware) {  
    $middleware->validateCsrfTokens(['/api/*']);  
})
```

Actividad 6.5

Implementa el método `postTweet()`. Recibirá el id del usuario que ha escrito el tweet, y el texto del tweet. La fecha se establecerá a la actual (usa la función `now()`).

3.3.3 Implementar PUT

Vamos a añadir la funcionalidad PUT a la ruta `/api/users/{id}`.

Para actualizar un usuario vamos a necesitar como mínimo los mismos parámetros que para el POST:

- **name:** nombre del usuario para modificar.
- **userName:** nombre de usuario del usuario para modificar. Debe ser único.

Vamos a implementar `putUser($id)`. Para ello tenemos que:

- Comprobar que el usuario a editar exista.
- Comprobar que, si se está intentando modificar el `userName`, el nuevo no esté ya en uso. En caso afirmativo, deberemos devolver un error 409 (Conflicto).
- Actualizar los atributos del usuario obtenido de la base de datos con la información aportada.
- Devolver un mensaje con código 201. Podemos devolver dos cosas:
 - Un objeto con la información.
 - Un enlace al objeto.

El código, devolviendo un enlace, quedaría de la siguiente forma:

```
#[Route('/api/users/{id}', name: 'api_put_user', methods: ['PUT'])]
function putUser($id, Request $request, ManagerRegistry $doctrine)
{
    $entityManager = $doctrine->getManager();
    $user = $entityManager->getRepository(User::class)->find($id);
    if ($user == null) {
        return new JsonResponse([
            'error' => 'User not found'
        ], 404);
    }
    //Si se intenta actualizar el userName, comprobar que en la BD no exista ya ese userName
    if ($user->getUserName() != $request->request->get("userName")) {
        $user2 = $entityManager->getRepository(User::class)->findOneBy(['userName' => $request->request->get("userName")]);
        if ($user2) {
            return new JsonResponse([
                'error' => 'UserName already in use'
            ], 409);
        }
    }
    $user->setName($request->request->get("name"));
    $user->setUserName($request->request->get("userName"));
    $entityManager->flush();
    return new JsonResponse($this->generateUrl('api_get_user', [
        'id' => $user->getId(),
    ], UrlGeneratorInterface::ABSOLUTE_URL));
}
```

Actividad 6.6

Implementa el método `putTweet($id, Request $request)`.

3.3.4 Implementar DELETE

Actividad 6.7

Con los métodos anteriores hechos, debes ser capaz de implementar los métodos `deleteUser($id)` y `deleteTweet($id)` por tu cuenta. Las claves son:

- Debes obtener el recurso (tweet o usuario) a partir de la id.
- Si el recurso no existe debes devolver un error código 404.
- Al borrar el recurso debes devolver una respuesta vacía (null) con código HTTP 204 (no content).